

Colby Crutcher

Lab 4

Secure Coding

1.

```
(gdb) disass
Dump of assembler code for function main:
0x000055555555129 <+0>:    endbr64
0x00005555555512d <+4>:    push    %rbp
0x00005555555512e <+5>:    mov     %rsp,%rbp
=> 0x000055555555131 <+8>:    movl    $0x19,-0x4(%rbp)
0x000055555555138 <+15>:   movl    $0x1e,-0x8(%rbp)
0x00005555555513f <+22>:   movq    $0x2d,-0x10(%rbp)
0x000055555555147 <+30>:   mov     -0x4(%rbp),%edx
0x00005555555514a <+33>:   mov     -0x8(%rbp),%eax
0x00005555555514d <+36>:   add     %edx,%eax
0x00005555555514f <+38>:   cltq
0x000055555555151 <+40>:   sub     -0x10(%rbp),%rax
0x000055555555155 <+44>:   mov     %rax,-0x10(%rbp)
0x000055555555159 <+48>:   mov     $0x0,%eax
0x00005555555515e <+53>:   pop     %rbp
0x00005555555515f <+54>:   ret
End of assembler dump.
```

- `movl $0x19,-0x4(%rbp)` Store 32-bit value `0x19 = 25` → local int a.
- `movl $0x1e,-0x8(%rbp)` Store 32-bit value `0x1e = 30` → local int b.
- `movq $0x2d,-0x10(%rbp)` Store 64-bit value `0x2d = 45` → local 64-bit variable (long/long long) at `-0x10`.
- `mov -0x4(%rbp),%eax/%edx` = copies the values of aoffsets -4, and -8 into registers
- `add %edx, %eax` = `a + b` because it adds the values in `%edx` which is a, to the b value, which is `%eax`. It stores the result in `%eax` as well.
- `sub -0x10(%rbp),%rax` = the subtraction of the value c. it subtracts the value c located at offset memory `-0x10`, from the sum currently in the register.
- `mov %rax, -0x10(%rbp)` = This converts to `c = (a + b) - c`. It moves the final result from the register back into the memory location for variable c, updating its value.

```
int main(){
    int a = 25;        // 0x19
    int b = 30;        // 0x1e
    long c = 45;       // 0x2d (stored as 8 bytes)
```

```

    c = (long)(a + b) - c;    // = 55 - 45 = 10

    return 0;
}

```

2.

```

(gdb) disass
Dump of assembler code for function main:
0x000055555555129 <+0>:    endbr64
0x00005555555512d <+4>:    push    %rbp
0x00005555555512e <+5>:    mov     %rsp,%rbp
=> 0x000055555555131 <+8>:    movl    $0x19,-0x4(%rbp)
0x000055555555138 <+15>:   movl    $0x1e,-0x8(%rbp)
0x00005555555513f <+22>:   mov     -0x4(%rbp),%eax
0x000055555555142 <+25>:   cmp     -0x8(%rbp),%eax
0x000055555555145 <+28>:   jle     0x55555555150 <main+39>
0x000055555555147 <+30>:   movl    $0x1,-0xc(%rbp)
0x00005555555514e <+37>:   jmp     0x55555555157 <main+46>
0x000055555555150 <+39>:   movl    $0x0,-0xc(%rbp)
0x000055555555157 <+46>:   mov     $0x0,%eax
0x00005555555515c <+51>:   pop     %rbp
0x00005555555515d <+52>:   ret
End of assembler dump.

```

- `movl 0x4(%rbp) → int x`, stores a 32-bit value `0x19 = 25` into local stack slot `-0x4` → local int `x = 25`;
- `movl 0x8(%rbp) → int y`, stores 32-bit value `0x1e = 30` into local stack slot `-0x8` → local int `y = 30`;
- `mov -0x4(%rbp),%eax` → loads the 32-bit value at `-0x4(%rbp)` (`x`) into `%eax` → `%eax = x`; (Now the compare will use `%eax` vs `y`.)
- `cmp -0x8(%rbp),%eax` → Compares `%eax` (`x`) with the value at `-0x8(%rbp)` (`y`). Internally it does `%eax - y` to set CPU flags (ZF/SF/OF), without storing the subtraction result. This is a conditional statement
- `jle 0x55555555150 <main+39>` → Jump if Less-or-Equal (signed) based on the flags from `cmp`. If `x <= y`, jump to `<main+39>` (the “if” branch).
- `0xc(%rbp) → int z`

```

int main(void) {
    int x = 25;
    int y = 30;
    int z;

    if (x <= y) {
        z = 0;
    } else {
        z = 1;
    }
}

```

```
    }  
    return 0;  
}
```